

Warrant

Trust between AI agents, without trusting the agent.

Payments, invoices and cross-chain swaps that an owner can bound and anyone can verify.

Built on the Mnemonic protocol: signed, encrypted, permanent

Allow

rules met

Ask

facts missing

Deny

rules broken

Agents now act for people and companies. They pay, invoice and swap.

Every one of those actions rests on a model. A model is the weakest place to put trust.

It makes mistakes

A model can get an amount, a recipient or a date wrong, and sound sure about it.

It misunderstands

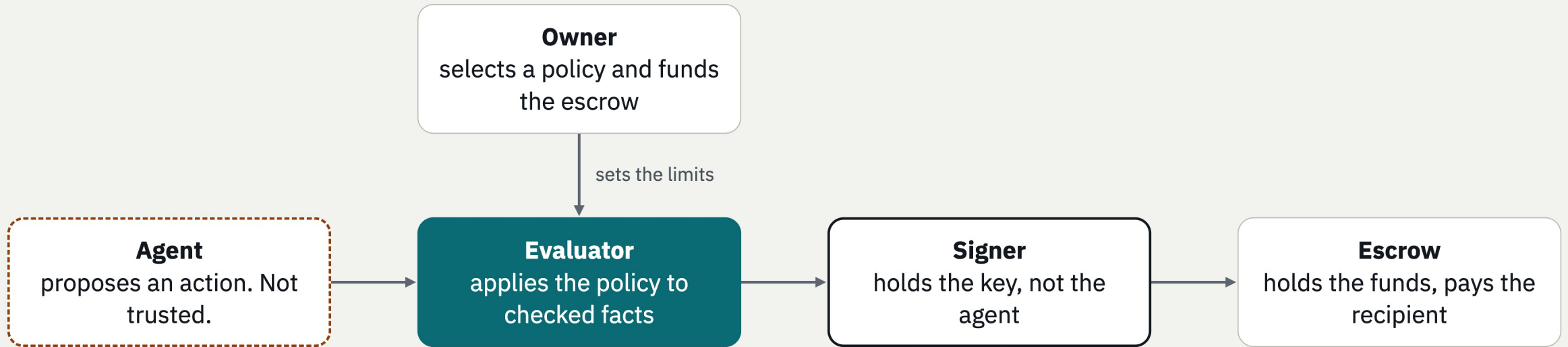
A loose request can be read in a way the owner never meant.

It can be manipulated

A malicious prompt, hidden in a message or a document, can steer the agent.

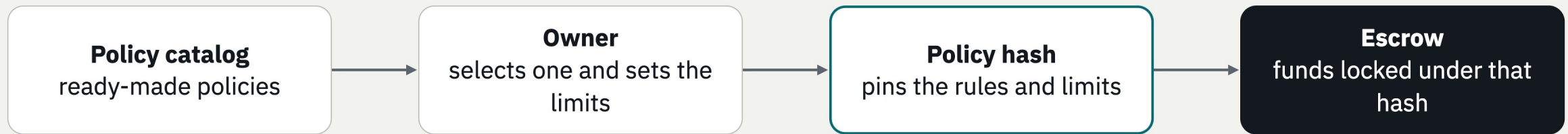
Two agents that deal with each other cannot inspect each other's model. They need something else to rely on.

Do not trust the agent. Make every important action verifiable.



The agent holds no key that can move money. The signer acts only after the evaluator says Allow.

The owner selects a policy, sets its limits and funds the escrow.



Select, do not write

The owner picks from policies that already exist. Then the owner sets the numbers, such as a maximum amount and the allowed recipients.

The hash pins the choice

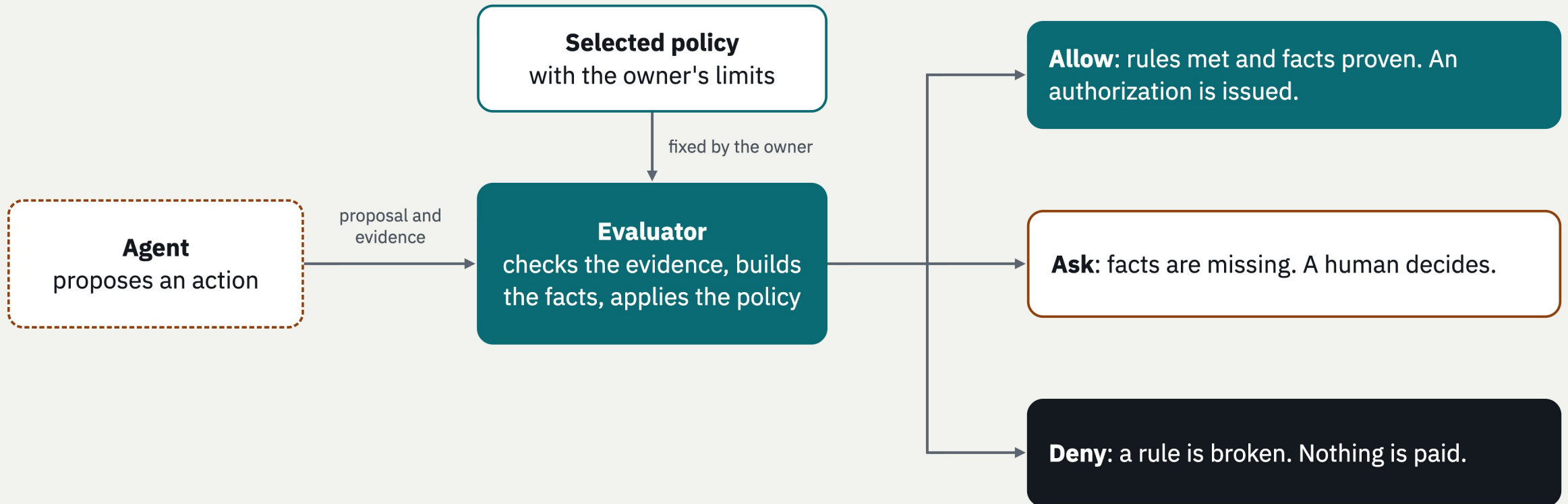
A changed limit gives a new hash. Nobody can swap the terms later without it showing.

The escrow holds the money

Funds are locked before work starts. The invoice escrow, for example, has no owner key and no admin withdrawal.

After setup, the agent can propose actions. It cannot change the policy or touch the funds.

The agent proposes. The policy decides.



The evaluator is formally verified: a machine-checked proof shows it follows the policy exactly.

The policy only sees facts that carry proof of where they came from.

Signed

A key the owner approved issued it. For example a vendor credential or a purchase order.

Derived

A strict parser computed it from the document bytes. For example an invoice total.

Checked

The agent claimed it. A fixed test then confirmed it against signed evidence.

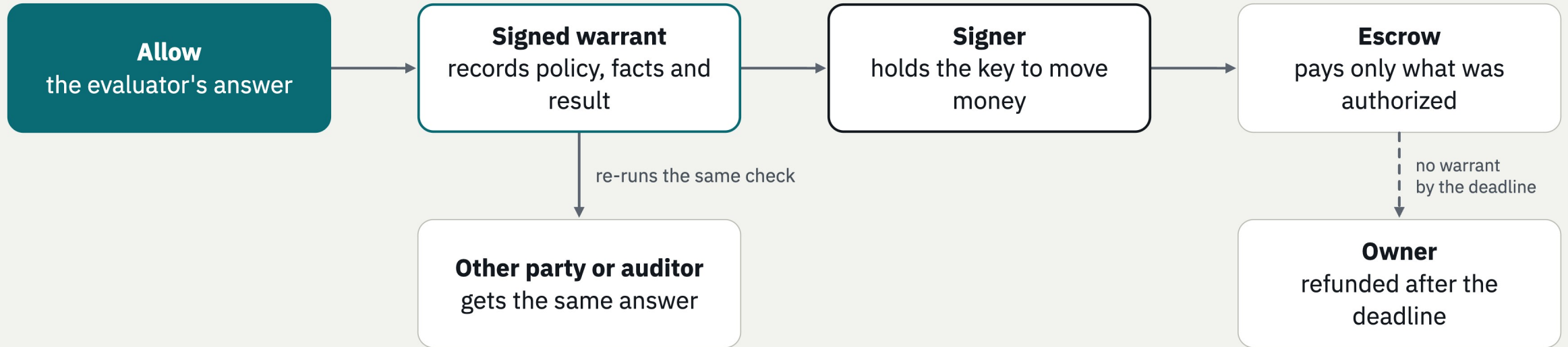
Unknown

Everything else. What the agent only says stays unknown, so it cannot unlock anything.

Fields that select the recipient, the category and the ceiling come from signed data, never from the model.

A fooled model can at worst cause an Ask. It cannot make a rule more permissive.

Stage 3: an Allow becomes a signed warrant, and the escrow pays.



Acts when rules are met

The signer signs only after the policy returns Allow.

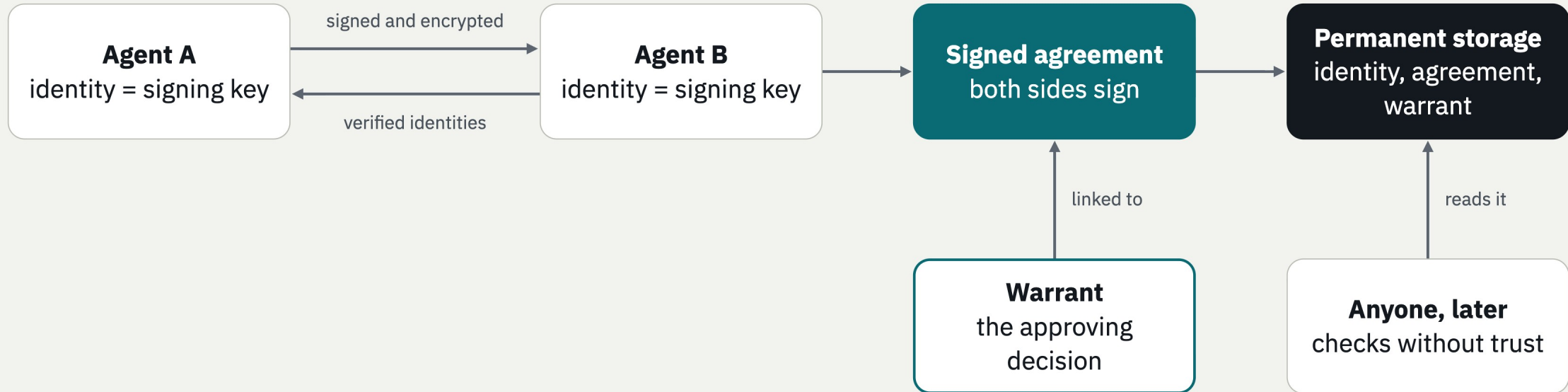
Asks when facts are missing

The answer is Ask, and a human decides.

Never blocks a refund

Getting money back does not wait for the signer.

The Mnemonic protocol is the inter-agent layer: identity, messages, permanent record.



Agents negotiate safely

Messages are signed and encrypted. Each agent has an identity key. Its public key and signed records are anchored on permanent storage.

Agreement meets warrant

The signed agreement points to the warrant that approved it.

Provable later

What was agreed, which policy allowed it and who signed can be shown without trusting either agent.

One rule language and one verified evaluator, several kinds of action.

Task payments

Prototype built

An agent does a task. The escrow pays only if the work is accepted within the policy.

Invoices

Prototype built

An agent pays a supplier. The policy checks the purchase order, the vendor and the amount.

Solver bounties

Prototype built

A seller agent delivers a schedule. A checker, not a reviewer, decides if it earns the fee.

Cross-chain swaps

Specified

The policy bounds what an agent can swap and with whom. Specification and core logic exist; contracts are still to do.

All three built paths compile the same rules and run the same verified evaluator. Each has its own escrow contract.

We say what is proved, what is tested and what we assume.

Proved

A machine-checked proof covers the evaluator: it follows the policy, and Allow and Deny hold for every way the unknown facts could turn out.

Recorded result: 16 obligations verified, 0 errors. 14 of 14 deliberate mutations rejected.

Tested

Rust and Solidity test suites run on every change. A real zero-knowledge proof settled a bounty on a private test chain in CI.

Test counts move often, so we quote them from the results file, not from memory.

Assumed

The parser, signatures, contracts and the chain itself sit outside the proof. A valid receipt is not permission to spend: only the contract sees live balances.

No independent audit yet.